DHT 구현 분석 문서

개요

Rain BitTorrent 클라이언트의 DHT(Distributed Hash Table) 구현은 BEP 5 표준을 따르며, 탈중앙화된 피어 발견 시스템을 제공합니다. DHT를 통해 트래커 없이도 피어를 찾을 수 있어 BitTorrent 네트워크의 견고성과 가용성을 크게 향상시킵니다.

주요 특징

- BEP 5 DHT 프로토콜 완전 구현
- 자동 부트스트래핑과 라우팅 테이블 관리
- 주기적 announce를 통한 피어 발견
- 프라이빗 토렌트에서는 DHT 비활성화
- IPv4 피어 주소 지원

DHT 노드 초기화와 부트스트래핑

1. DHT 설정 구조

DHT 관련 설정은 `Config` 구조체에 정의되어 있습니다:

```
type Config struct {
// DHT 활성화 여부
DHTEnabled  bool  `yaml:"dht-enabled"`
// DHT 노드가 바인딩할 IP 주소
```

```go
DHTHost string `yaml:"dht-host"`
// DHT 노드가 사용할 UDP 포트
DHTPort uint16 `yaml:"dht-port"`
// DHT announce 간격
DHTAnnounceInterval time.Duration `yaml:"dht-announce-interval"`
// 최소 DHT announce 간격 (피어가 부족할 때)
DHTMinAnnounceInterval time.Duration `yaml:"dht-min-announce-interval"`
// 부트스트랩에 사용할 알려진 DHT 라우터 목록
DHTBootstrapNodes []string `yaml:"dht-bootstrap-nodes"`
}
```

2. DHT 노드 초기화 과정

Session 생성 시 DHT 노드가 초기화됩니다 (`session.go`):

```go
func NewSession(cfg Config) (*Session, error) {
// ... 기타 초기화 코드 ...
var dhtNode *dht.DHT
if cfg.DHTEnabled {
// DHT 설정 생성
dhtConfig := dht.NewConfig()
dhtConfig.Address = cfg.DHTHost
dhtConfig.Port = int(cfg.DHTPort)
dhtConfig.DHTRouters = strings.Join(cfg.DHTBootstrapNodes, ",")
```

```
dhtConfig.SaveRoutingTable = false

dhtConfig.NumTargetPeers = 0

// DHT 노드 생성

dhtNode, err = dht.New(dhtConfig)

if  err != nil {

return  nil, err

}

// DHT 노드 시작

err = dhtNode.Start()

if  err != nil {

return  nil, err

}

}

// ... Session 구조체에 dhtNode 할당 ...

}
```

3. 부트스트래핑 과정

DHT 노드는 시작 시 다음과 같은 부트스트래핑 과정을 거칩니다:

1. **부트스트랩 노드 연결**: `DHTBootstrapNodes` 에 설정된 알려진 DHT 라우터들에 연결
2. **라우팅 테이블 구축**: 부트스트랩 노드들로부터 다른 노드 정보를 수집하여 라우팅 테이블 구성
3. **노드 ID 생성**: 160비트 노드 ID를 무작위로 생성
4. **네트워크 참여**: DHT 네트워크에 참여하여 쿼리 응답 시작

피어 발견 과정

1. DHT 피어 요청 메커니즘

DHT를 통한 피어 발견은 다음과 같은 구조로 관리됩니다:

```
type Session struct {
dht *dht.DHT
dhtPeerRequests  map[*torrent]struct{}
mPeerRequests sync.Mutex
// ...
}
```

2. 피어 요청 프로세스

토렌트가 더 많은 피어를 필요로 할 때 DHT에 피어를 요청합니다:

```
// torrent.go - DHT에 피어 요청
func (t *torrent) announceDHT() {
t.session.mPeerRequests.Lock()
t.session.dhtPeerRequests[t] = struct{}{}
t.session.mPeerRequests.Unlock()
}
```

3. DHT 결과 처리

Session은 별도 고루틴에서 DHT 결과를 처리합니다 (`session_dht.go`):

```
func (s *Session) processDHTResults() {
dhtLimiter := time.NewTicker(time.Second)
defer dhtLimiter.Stop()
for {
select {
case <-dhtLimiter.C:
s.handleDHTtick()
case res := <-s.dht.PeersRequestResults:
// DHT에서 받은 피어 정보 처리
for ih, peers := range res {
s.mTorrents.RLock()
torrents, ok := s.torrentsByInfoHash[ih]
s.mTorrents.RUnlock()
if !ok {
continue
}
// 피어 주소 파싱
addrs := parseDHTPeers(peers)
// 해당 토렌트들에 피어 정보 전달
for _, t := range torrents {
select {
```

```
case t.torrent.dhtPeersC <- addrs:
case <-t.torrent.closeC:
default:
}
}
}
case <-s.closeC:
return
}
}
}
```

4. DHT 피어 주소 파싱

DHT에서 받은 피어 정보를 TCP 주소로 변환합니다:

```
func parseDHTPeers(peers []string) []*net.TCPAddr {
addrs := make([]*net.TCPAddr, 0, len(peers))
for _, peer := range peers {
if len(peer) != 6 {
// 현재는 IPv4만 지원
continue
}
// 6바이트 형식: IP(4바이트) + Port(2바이트)
```

```
addr := &net.TCPAddr{
IP: net.IP(peer[:4]),
Port: int((uint16(peer[4]) << 8) | uint16(peer[5])),
}
addrs = append(addrs, addr)
}
return addrs
}
```

DHT Announce 구현

1. DHT Announcer 구조

DHT announce는 전용 announcer를 통해 관리됩니다 (`internal/announcer/dht.go`):

```
type DHTAnnouncer struct {
lastAnnounce time.Time
needMorePeers bool
needMorePeersC chan bool
closeC chan struct{}
doneC chan struct{}
}
```

2. DHT Announcer 생성과 실행

토렌트 시작 시 DHT announcer가 생성됩니다 (`torrent_start.go`):

```
func (t *torrent) start() {
// ... 기타 시작 로직 ...
// DHT announcer 생성 (프라이빗 토렌트가 아닌 경우에만)
if  t.dhtAnnouncer == nil &&
t.session.config.DHTEnabled &&
(t.info == nil || !t.info.Private) {
t.dhtAnnouncer = announcer.NewDHTAnnouncer()
// DHT announce 고루틴 시작
go  t.dhtAnnouncer.Run(
t.announceDHT,
t.session.config.DHTAnnounceInterval,
t.session.config.DHTMinAnnounceInterval,
t.log,
)
}
}
```

3. 주기적 Announce 로직

DHT announcer는 다음 로직으로 주기적 announce를 수행합니다:

```
func (a *DHTAnnouncer) Run(announceFunc  func(), interval, minInterval time.Dura
defer  close(a.doneC)
```

```
timer := time.NewTimer(minInterval)

defer  timer.Stop()

// 타이머 리셋 함수

resetTimer := func() {

if  a.needMorePeers {

// 피어가 부족하면 최소 간격 사용

timer.Reset(time.Until(a.lastAnnounce.Add(minInterval)))

} else {

// 일반적인 경우 표준 간격 사용

timer.Reset(time.Until(a.lastAnnounce.Add(interval)))

}

}

// announce 함수

announce := func() {

announceFunc()

a.lastAnnounce = time.Now()

resetTimer()

}

// 초기 announce 수행

announce()

// 메인 루프

for {

select {
```

```
case <-timer.C:
announce()
case  a.needMorePeers = <-a.needMorePeersC:
resetTimer()
case <-a.closeC:
return
}
}
}
```

4. 피어 부족 신호 처리

토렌트가 더 많은 피어를 필요로 할 때 DHT announcer에 신호를 보냅니다:

```
// torrent_peer.go
func (t *torrent) needMorePeers(val  bool) {
// ... 트래커 announcer 처리 ...
if  t.dhtAnnouncer != nil {
t.dhtAnnouncer.NeedMorePeers(val)
}
}
```

라우팅 테이블 관리

1. DHT 틱 처리

Session은 1초마다 DHT 틱을 처리하여 대기 중인 피어 요청을 처리합니다:

```
func (s *Session) handleDHTtick() {
s.mPeerRequests.Lock()
defer s.mPeerRequests.Unlock()
// 대기 중인 피어 요청 중 하나를 처리
for t := range s.dhtPeerRequests {
// DHT에 피어 요청 (포트 정보 포함)
s.dht.PeersRequestPort(string(t.infoHash[:]), true, t.port)
delete(s.dhtPeerRequests, t)
return // 한 번에 하나씩만 처리
}
}
```

2. DHT 노드 정보 추가

피어로부터 받은 포트 메시지를 통해 DHT 노드 정보를 추가합니다:

```
// torrent_messagehandler.go
func (t *torrent) handlePeerMessage(pe *peer.Peer, msg peerprotocol.Message) {
switch msg := msg.(type) {
// ... 기타 메시지 처리 ...
case peerprotocol.PortMessage:
```

```
// 피어가 DHT 포트 정보를 알려줌
if  t.session.dht != nil {
t.session.dht.AddNode(fmt.Sprintf("%s:%d", pe.IP(), msg.Port))
}
}
}
```

3. InfoHash 관리

토렌트 제거 시 DHT에서도 해당 InfoHash를 제거합니다:

```
// session.go
func (s *Session) removeTorrent(ih [20]byte) {
// ... 토렌트 제거 로직 ...
// 해당 InfoHash를 사용하는 토렌트가 더 이상 없으면 DHT에서 제거
if  s.config.DHTEnabled && len(s.torrentsByInfoHash[ih]) == 0 {
s.dht.RemoveInfoHash(string(ih))
}
}
```

DHT와 피어 연결 통합

1. 피어 소스 분류

DHT를 통해 발견된 피어는 `peersource.DHT` 로 분류됩니다:

```go
// peersource/peersource.go
const (
Tracker Source = iota
DHT  // DHT를 통해 발견된 피어
PEX
Manual
Incoming
)
```

2. DHT 피어 처리

토렌트는 DHT에서 받은 피어 정보를 처리합니다:

```go
// torrent_run.go - 토렌트 메인 루프
func (t *torrent) run() {
for {
select {
// ... 기타 케이스들 ...
case  addrs := <-t.dhtPeersC:
// DHT에서 받은 피어들을 DHT 소스로 처리
t.handleNewPeers(addrs, peersource.DHT)
}
}
```

```
}
```

3. DHT 통계 정보

DHT를 통해 발견된 피어 수는 별도로 추적됩니다:

```
// torrent_stats.go
type Stats struct {
Addresses  struct {
Total  int
Tracker  int
DHT  int  // DHT를 통해 발견된 피어 주소 수
PEX  int
}
}

func (t *torrent) Stats() Stats {
// ...
s.Addresses.DHT = t.addrList.LenSource(peersource.DHT)
// ...
}
```

성능 고려사항

1. DHT 요청 제한

DHT 피어 요청은 1초에 하나씩만 처리하여 DHT 네트워크에 과부하를 주지 않습니다:

```
func (s *Session) processDHTResults() {
dhtLimiter := time.NewTicker(time.Second) // 1초 간격 제한
defer dhtLimiter.Stop()
// ...
}
```

2. 프라이빗 토렌트 제외

프라이빗 토렌트는 DHT를 사용하지 않아 트래커 정책을 준수합니다:

```
if t.dhtAnnouncer == nil &&
t.session.config.DHTEnabled &&
(t.info == nil || !t.info.Private) { // 프라이빗 토렌트 제외
// DHT announcer 생성
}
```

3. 메모리 효율성

DHT 설정에서 라우팅 테이블 저장을 비활성화하여 메모리 사용량을 줄입니다:

```
dhtConfig.SaveRoutingTable = false
dhtConfig.NumTargetPeers = 0
```

관련 문서

선행 지식

- 01-system-architecture.md - DHT가 전체 시스템에서 차지하는 위치
- 03-01-torrent-lifecycle-process.md - DHT announcer 생성과 관리

관련 주제

- 05-02-pex-protocol-analysis.md - 다른 피어 발견 메커니즘과의 비교
- 04-01-networking-packages-analysis.md - 피어 소스 관리와 연결

후속 학습

- 06-01-performance-optimization-analysis.md - DHT 성능 최적화 기법
- 07-01-library-usage-guide.md - DHT 설정 및 사용법

> **네비게이션:** 이전: 네트워킹 패키지 분석 | 목차 | 다음: PEX 프로토콜 분석